ALE-GOV-RIGHTS-TREATY-GRAMMAR-001.aln

Canonical ALN Rights and Treaty Grammar for Aletheion

Destination: `aletheion/governance/aln/ALE-GOV-RIGHTS-TREATY-GRAMMAR-001.aln`

Language: ALN (Aletheion Legal Norm)

Purpose: Define machine-enforceable grammar atoms for rights, treaties, and obligations that all 25 workflows import as compile-time governance primitives. Prevents workflow-specific treaty drift and ensures uniform interpretation across Edge-Rust-Model (ERM) layers, Birth-Signs binding, optimization engines, compliance preflight pipelines, and Googolswarm trust-layer append operations.

Scope: Legal norms, Indigenous protocols (FPIC), BioticTreaties (cross-species), MicroTreaties (LexEthos neighborhood/workplace), and CryptoSomatic consent envelopes for biosignal governance.

Integration Points:

- Birth-Signs module (ALE-GOV-BIRTH-SIGN-MODEL-001.rs) references `RightsAtom`, `BioticTreaty`, `MicroTreaty` types
- Googolswarm provenance schema (ALE-TRUST-GOVERNED-DECISION-TX-001.aln) embeds `AlnNormId` lists
- LexEthos rights compiler (`aletheion/governance/lexethos`) imports these atoms
- Synthexis model (`aletheion/synthesis/model`) uses BioticTreaty constraints
- CryptoSomatic Shield (`aletheion/health/augmentation`) enforces consent state machines via RightsAtom modalities

---

Part I: Core Governance Atoms

1.1 RightsAtom

A **RightsAtom** is the minimal unit of a legal or ethical obligation, structured as a quintuple: **(Holder, Object, Modality, Conditions, Expiry)**.

```
type RightsAtom {
holder: Did | CollectiveId | SpeciesAgent | "Public";
object: ResourceRef | ActionType | DataCategory;
modality: Obligation | Permission | Prohibition;
conditions: Condition[];
expiry: Timestamp | "Perpetual";
enforcement_mode: "Hard" | "HighPenalty" | "Advisory";
provenance: ProvenanceRef;
}
```

Holder Types:

- `Did`: Individual Digital Identity (citizen, augmented-citizen, device)
- `CollectiveId`: Indigenous nation, neighborhood collective, species-protection organization
- `SpeciesAgent`: Non-human lifeform representation (e.g., Sonoran Desert tortoise, Gila River fish populations)
- `"Public"`: Universal holder (applies to all actors in jurisdiction)

Object Categories:

- `ResourceRef`: Water allocation node, heat budget tile, energy microgrid, biosignal stream
- `ActionType`: Routing decision, sensor deployment, augmentation activation, freight movement
- `DataCategory`: Location trace, biometric envelope, carbon footprint, somatic state

Modality Semantics:

- `Obligation`: Actor MUST perform action or respect constraint (hard enforcement default)

- Permission: Actor MAY perform action if conditions satisfied (opt-in)
- Prohibition: Actor MUST NOT perform action (absolute block unless emergency override with audit trail)

Conditions Grammar:

```
type Condition {
predicate: String; // ALN-compatible logical expression
parameters: Map<String, Value>;
birth_sign_scoped: Bool; // True if condition only applies within specific BirthSignId tiles
}
```

Enforcement Mode:

- Hard: Violation causes immediate rejection at governance preflight (Stage 5)
- HighPenalty: Violation adds large cost penalty to optimization objective function (Stage 4)
- Advisory: Logged but not enforced; triggers citizen grievance surface notification (Stage 7)

Example RightsAtom (Water Access for Indigenous Territory):

```
RightsAtom {
holder: CollectiveId("Akimel_OOdham_Nation"),
object: ResourceRef("colorado_river_allocation_senior_rights"),
modality: Obligation,
conditions: [
{predicate: "allocation_priority >= 1", parameters: {},
birth_sign_scoped: true},
{predicate: "fpic_status == Granted", parameters: {territory:
"Akimel_OOdham"}, birth_sign_scoped: true}
],
expiry: Perpetual,
enforcement_mode: "Hard",
provenance: ProvenanceRef("Treaty_1924_PimaMaricopa")
}
```

1.2 BioticTreaty

A **BioticTreaty** encodes cross-species obligations: habitat corridors, light/noise limits, pesticide restrictions, thermal refugia requirements. These treaties bind human actions to ecological stewardship norms.

```
type BioticTreaty {
treaty_id: BioticTreatyId;
species_agents: SpeciesAgent[];
domain: "Water" | "Thermal" | "Light" | "Noise" | "Chemical" |
"Habitat" | "Air";
constraints: BioticConstraint[];
habitat_corridors: GeometryRef[];
enforcement_mode: "Hard" | "HighPenalty";
effective_date: Timestamp;
review_cycle: Duration;
provenance: ProvenanceRef;
}

type BioticConstraint {
metric: String; // e.g., "nocturnal_light_intensity_lux",
"riparian_corridor_width_m"
threshold: Value;
operator: "<" | ">" | "<=" | ">=" | "==";
spatial_scope: TileSet | "Global";
temporal_scope: TimeWindow | "Continuous";
}

type SpeciesAgent {
species_name: String;
iucn_status: "LC" | "NT" | "VU" | "EN" | "CR" | "EW" | "EX";
indigenous_name: String | Null;
representation_body: Did | CollectiveId;
}
```

Domain-Specific Constraints:

- **Water**: Minimum riparian flow rates, seasonal flood pulse requirements
- **Thermal**: Maximum urban heat island delta relative to natural baseline

- **Light**: Scotopic lux limits during nesting/migration seasons
- **Noise**: dB(A) thresholds for breeding habitats
- **Chemical**: Pesticide/herbicide exclusion zones
- **Habitat**: Corridor width, connectivity indices, barrier-free passage requirements
- **Air**: Particulate matter limits in sensitive ecological zones

Example BioticTreaty (Sonoran Desert Tortoise Light Protection):

```
BioticTreaty {
treaty_id:
BioticTreatyId("Phoenix_Tortoise_Scotopic_Protection_2026"),
species_agents: [
{
species_name: "Gopherus morafkai",
iucn_status: "VU",
indigenous_name: "Komkçĕḍ" (O'odham),
representation_body: CollectiveId("Sonoran_Desert_Conservancy")
}
],
domain: "Light",
constraints: [
{
metric: "nocturnal_scotopic_lux",
threshold: 0.5,
operator: "<=",
spatial_scope: TileSet(["tortoise_habitat_zone_A",
"tortoise_habitat_zone_B"]),
temporal_scope: TimeWindow("21:00-05:00", "March-October")
}
],
habitat_corridors:
[GeometryRef("south_mountain_preserve_corridor")],
enforcement_mode: "Hard",
effective_date: 2026-04-01T00:00:00Z,
review_cycle: Duration("2_years"),
provenance:
ProvenanceRef("City_of_Phoenix_BioticTreaty_Ordinance_2026_12")
}
```

1.3 MicroTreaty

A **MicroTreaty** represents hyper-local governance agreements: neighborhood cooling norms, workplace augmentation boundaries, school pedestrian safety protocols, event-specific mobility rules. These are citizen-defined or institution-defined overlays that refine city-wide policy.

```
type MicroTreaty {
treaty_id: MicroTreatyId;
scope: NeighborhoodId | WorkplaceId | FacilityId | EventId;
initiator: Did | CollectiveId;
domain: "Cooling" | "Mobility" | "Noise" | "Augmentation" |
"DataUse" | "Energy";
rules: MicroRule[];
consensus_mechanism: "LiquidDemocracy" | "DirectVote" |
"Delegated";
approval_threshold: f64; // 0.0-1.0
active_period: TimeWindow | "Indefinite";
dispute_resolution: DisputeProtocol;
enforcement_mode: "HighPenalty" | "Advisory";
provenance: ProvenanceRef;
}

type MicroRule {
rule_id: String;
statement: String; // Human-readable policy text
aln_constraint: Condition; // Machine-executable translation
affected_assets: AssetId[];
override_priority: u8; // Lower = higher priority (0 = absolute)
}

type DisputeProtocol {
protocol_name: String;
mediation_body: CollectiveId | Did;
escalation_path: String[];
cooling_period: Duration;
}
```

Use Cases:

- **Cooling MicroTreaty**: "Roosevelt Neighborhood requests max outdoor temp differential of 3°C vs. natural shade baseline during 14:00-18:00 heatwaves"
- **Mobility MicroTreaty**: "Downtown Arts District reduces freight routing 50% during First Friday art walk events"
- **Augmentation MicroTreaty**: "Hospital campus prohibits neural recording devices in patient-care wings without explicit per-visit consent"
- **DataUse MicroTreaty**: "University campus requires anonymized-only pedestrian flow analytics; no persistent DID tracking"

Example MicroTreaty (Neighborhood Shade Equity):

```
MicroTreaty {
treaty_id: MicroTreatyId("Roosevelt_Shade_Equity_2026"),
scope: NeighborhoodId("Roosevelt_Historic_District"),
initiator: CollectiveId("Roosevelt_Neighborhood_Council"),
domain: "Cooling",
rules: [
{
rule_id: "shade_allocation_priority",
statement: "Residential blocks with >60% heat-vulnerable population
receive priority shade structure deployment",
aln_constraint: {
predicate: "heat_vulnerability_index > 0.6 IMPLIES
shade_allocation_priority >= 8",
parameters: {},
birth_sign_scoped: true
},
affected_assets: ["thermaphora_tile_grid_roosevelt"],
override_priority: 2
}
],
consensus_mechanism: "LiquidDemocracy",
approval_threshold: 0.67,
active_period: "Indefinite",
dispute_resolution: {
protocol_name: "DisputeCoolingProtocol_v1",
mediation_body: CollectiveId("Phoenix_Heat_Equity_Board"),
escalation_path: ["Neighborhood_Council", "City_Heat_Office",
```

"Mayor_Review"],
cooling_period: Duration("14_days")
},
enforcement_mode: "HighPenalty",
provenance:
ProvenanceRef("Roosevelt_Treaty_Registry_2026_03_08")
}

1.4 FpicRequirement

Free, Prior, and Informed Consent (FPIC) is the governance primitive for Indigenous sovereignty. Any action intersecting recognized Indigenous territories MUST evaluate FPIC status before proceeding.

```
type FpicRequirement {
requirement_id: FpicRequirementId;
territory: IndigenousTerritoryId;
nation: String;
traditional_name: String;
requirement_level: "NotApplicable" | "RequiredBeforePlanning" |
"RequiredBeforeExecution" | "EmergencyOverrideWithAudit";
protocols: FpicProtocol[];
contact_authority: Did | CollectiveId;
provenance: ProvenanceRef;
}

type FpicProtocol {
protocol_name: String;
stages: String[]; // e.g., ["Notification", "Consultation", "Negotiation",
"Consent"]
minimum_duration: Duration;
documentation_required: String[];
}

type FpicGrant {
grant_id: FpicGrantId;
requirement_ref: FpicRequirementId;
action_description: String;
granted_by: Did | CollectiveId;
```

```
granted_at: Timestamp;
expires_at: Timestamp | "Indefinite";
conditions: Condition[];
audit_trail: ProvenanceRef[];
}

type FpicStatus {
status: "NotRequired" | "Granted" | "Pending" | "Denied";
details: FpicGrant | FpicPendingRecord | FpicDenialRecord | Null;
}

type FpicPendingRecord {
territory: IndigenousTerritoryId;
requested_at: Timestamp;
expected_response_by: Timestamp;
}

type FpicDenialRecord {
territory: IndigenousTerritoryId;
denied_at: Timestamp;
reason: String;
appeal_path: String[];
}
```

FPIC Integration with Birth-Signs:

- Every Birth-Sign for tiles intersecting Indigenous territories includes `FpicRequirement` records
- Optimization engines (Stage 4) flag actions requiring FPIC as PendingFpic outcome
- Governance preflight (Stage 5) blocks actuation until `FpicStatus == Granted`
- Trust-layer append (Stage 3) records FPIC consultation timeline for audit transparency

Example FpicRequirement (Salt River Pima-Maricopa Indian Community):

```
FpicRequirement {
requirement_id:
FpicRequirementId("SRPMIC_Water_Infrastructure_FPIC"),
territory:
```

```
IndigenousTerritoryId("Salt_River_Pima_Maricopa_Reservation"),
nation: "Salt River Pima-Maricopa Indian Community (SRPMIC)",
traditional_name: "Onk Akimel O'odham / Xalychidom Piipaash",
requirement_level: "RequiredBeforePlanning",
protocols: [
{
protocol_name: "SRPMIC_Water_Consultation_Protocol_2024",
stages: ["Notification", "Technical_Review",
"Community_Consultation", "Tribal_Council_Vote"],
minimum_duration: Duration("90_days"),
documentation_required: ["Environmental_Impact_Assessment",
"Water_Rights_Analysis", "Cultural_Heritage_Survey"]
}
],
contact_authority:
CollectiveId("SRPMIC_Water_Resources_Department"),
provenance:
ProvenanceRef("SRPMIC_Treaty_1879_ExecutiveOrder_1924")
}
```

Part II: Cross-Domain Grammar Extensions

2.1 CryptoSomatic Consent Atoms

For biosignal governance (augmentation, neural interfaces, somatic data), consent is managed via state machines with explicit revocation paths.

```
type ConsentAtom {
consent_id: ConsentId;
subject: Did;
object: BiosignalCategory | AugmentationDeviceId;
modality: Permission;
conditions: Condition[];
granted_at: Timestamp;
expires_at: Timestamp | "UntilRevoked";
revocation_path: RevocationProtocol;
cryptosomatic_envelope: SomaticEnvelope;
}
```

```
type BiosignalCategory {
category: "Neural" | "Cardiac" | "Metabolic" | "Kinematic" |
"Environmental";
sensitivity_level: "Public" | "Aggregated" | "Pseudonymized" |
"Identifiable";
retention_policy: Duration | "SessionOnly";
}

type SomaticEnvelope {
fear_threshold: f64; // 0.0-1.0
pain_threshold: f64;
sanity_threshold: f64; // Cognitive load / attentional exhaustion
enforcement: "PreemptiveShutdown" | "Alert" | "Log";
}

type RevocationProtocol {
mechanism: "Immediate" | "GracePeriod" | "SessionEnd";
grace_duration: Duration | Null;
fallback_action: "DeleteData" | "Anonymize" | "Quarantine";
}
```

Integration with Neurorights Guardrail:

- `SomaticAnomalyDetector` monitors biosignals against `SomaticEnvelope` thresholds
- Violations trigger `AugmentationSafetyKernel` to enforce `PreemptiveShutdown` or derating
- Consent revocation propagates through all workflow stages within 500ms (hard real-time requirement)

Example ConsentAtom (Neural Interface for Mobility Assistance):

```
ConsentAtom {
consent_id:
ConsentId("User_12948_Neural_Mobility_Consent_2026_03_08"),
subject:
Did("did:bostrom:18sd2ujv24ual9c9pshtxys6j8knh6xaead9ye7"),
object: AugmentationDeviceId("Neuralink_N1_Unit_8472"),
modality: Permission,
conditions: [
{predicate: "usage_context == 'mobility_assistance_only'",
parameters: {}, birth_sign_scoped: false},
```

```
{predicate: "data_retention <= 24_hours", parameters: {},
birth_sign_scoped: false}
],
granted_at: 2026-03-08T13:39:00Z,
expires_at: "UntilRevoked",
revocation_path: {
mechanism: "Immediate",
grace_duration: Null,
fallback_action: "DeleteData"
},
cryptosomatic_envelope: {
fear_threshold: 0.7,
pain_threshold: 0.6,
sanity_threshold: 0.8,
enforcement: "PreemptiveShutdown"
}
}
```

2.2 Provenance and Audit Linking

Every governance atom MUST include `ProvenanceRef` linking to verifiable source documents (treaties, ordinances, community votes, scientific studies).

```
type ProvenanceRef {
source_type: "Treaty" | "Ordinance" | "CommunityVote" |
"ScientificStudy" | "ExecutiveOrder";
source_id: String;
document_hash: String; // Content-addressed hash for immutability
publication_date: Timestamp;
verification_url: String | Null;
multisig_attestors: Did[]; // DIDs of authorities who signed/verified
}
```

Googolswarm Integration:

- All `ProvenanceRef` records append to blockchain ledger at governance atom registration time
- Multi-sig attestation ensures cryptographic verification of source authenticity

- Audit trails query provenance to reconstruct decision lineage across workflow stages

---

Part III: Constraint Compilation and Enforcement

3.1 Constraint Propagation Rules

Birth-Signs attach governance atoms to spatial tiles. Workflows propagate these constraints through all 7 stages.

Stage-Specific Enforcement:

Stage	Constraint Role	ALN Grammar Used
S1 (Edge Ingestion)	Attach BirthSignId metadata	N/A (identity binding)
S2 (State Model Update)	Preserve governance context	RightsAtom, BioticTreaty refs
S3 (Trust Append)	Log ALN norms consulted	ProvenanceRef, FpicStatus
S4 (Optimization)	Add treaty constraints to objective	BioticConstraint, MicroRule
S5 (ALN Enforcement)	Validate all atoms, reject violations	All grammar types
S6 (Actuation)	Execute only approved actions	N/A (enforcement complete)
S7 (Citizen Surface)	Display applicable treaties/rights	Human-readable translations

Table 1: Constraint propagation through canonical 7-stage workflow spine

Optimization Engine Integration:

For NSGA-II/MOEA-D multi-objective optimizers, constraints become:

$$\text{Hard Constraint: } g_i(\mathbf{x}) \leq 0 \implies \text{Reject if violated}$$

$$\text{High-Penalty: } f_{\text{penalty}}(\mathbf{x}) = \sum_j w_j \cdot \max(0, g_j(\mathbf{x}))^2$$

Where:

- $g_i(\mathbf{x})$: Constraint function derived from `BioticConstraint` or `MicroRule`
- w_j: Penalty weight (10^3-10^6 for HighPenalty enforcement)
- $\mathbf{x}$: Decision vector (e.g., water allocations, routing paths, heat budgets)

Example Constraint Translation (Light Treaty → Objective Function):

```
BioticConstraint {
metric: "nocturnal_scotopic_lux",
threshold: 0.5,
operator: "<=",
spatial_scope: TileSet(["zone_A"]),
temporal_scope: TimeWindow("21:00-05:00")
}
```

Becomes optimizer constraint:

$$g_{\text{light}}(\mathbf{x}) = \text{scotopic_lux}(\mathbf{x}, \text{zone_A}, t) - 0.5 \leq 0 \quad \forall t \in [21{:}00,$$

If `enforcement_mode == "Hard"`: Reject any solution violating g_{light}.

If `enforcement_mode == "HighPenalty"`: Add penalty term:

$$f_{\text{penalty}} = 10^6 \cdot \max(0, g_{\text{light}}(\mathbf{x}))^2$$

3.2 Compliance Preflight Integration

The centralized compliance runtime (`ALE-COMP-CORE-001.rs`) validates all governance atoms before actuation.

Validation Steps:

1. **BirthSign Lookup:** Query geospatial DB for applicable BirthSignId(s) based on action's spatial footprint

2. **Atom Retrieval:** Fetch all RightsAtom, BioticTreaty, MicroTreaty, FpicRequirement records linked to BirthSign
3. **Condition Evaluation:** Execute ALN predicates against current state model snapshot
4. **FPIC Check:** If any FpicRequirement applies, verify FpicStatus == Granted
5. **Neurorights Check:** For biosignal/augmentation actions, validate ConsentAtom and SomaticEnvelope thresholds
6. **Aggregation:** Return GovernanceEvaluation with outcomes list and enforcement mode
7. **Decision Derivation:** Map evaluation to DecisionOutcome (Approved / Rejected / PendingFpic)

Pseudo-code Integration (Rust):

```
fn governance_preflight_check(
action: &ProposedAction,
birth_signs: &[BirthSignId],
state_model: &StateModelSnapshot,
) -> Result<GovernanceEvaluation, ComplianceError> {
let mut outcomes = Vec::new();
let mut fpic_status = None;

for birth_sign_id in birth_signs {
    let birth_sign = fetch_birth_sign(birth_sign_id)?;

    // Check RightsAtoms
    for rights_atom in &birth_sign.rights_atoms {
        let outcome = evaluate_rights_atom(rights_atom, action, state_model);
        outcomes.push(outcome);
    }

    // Check BioticTreaties
    for treaty in &birth_sign.biotic_treaties {
        let outcome = evaluate_biotic_treaty(treaty, action, state_model);
        outcomes.push(outcome);
    }

    // Check FPIC
```

```
    if let Some(fpic_req) = &birth_sign.fpic_requirement {
        fpic_status = Some(check_fpic_status(fpic_req, action)?);
    }
}

Ok(GovernanceEvaluation {
    birth_sign_id: birth_signs[0].clone(),
    constraint_mode: determine_strictest_mode(&outcomes),
    outcomes,
    fpic_status,
})
```

}

Part IV: Registry and Lifecycle Management

4.1 Grammar Registry

All governance atoms stored in versioned, immutable ledger with update audit trail.

```
type GovernanceRegistry {
rights_atoms: Map<AlnNormId, RightsAtom>;
biotic_treaties: Map<BioticTreatyId, BioticTreaty>;
micro_treaties: Map<MicroTreatyId, MicroTreaty>;
fpic_requirements: Map<FpicRequirementId, FpicRequirement>;
consent_atoms: Map<ConsentId, ConsentAtom>;
version: u32;
last_updated: Timestamp;
provenance_ledger: ProvenanceRef[];
}
```

Update Protocol:

1. Propose new atom or amendment via governance workflow (liquid democracy)
2. Undergo public comment period (minimum 14 days for BioticTreaties, 7 days for MicroTreaties)

3. Multi-sig approval from relevant authorities (City Council, Tribal Council, Neighborhood Council)
4. Append to Googolswarm with `ProvenanceRef` and attestor DIDs
5. Increment registry version, deploy to all workflow instances within 24 hours
6. Notify affected citizens via Stage 7 citizen surface

Deprecation Rules:

- Atoms can be marked `Deprecated` but never deleted (audit immutability)
- Deprecated atoms remain queryable for historical decision reconstruction
- New actions MUST NOT reference deprecated atoms (enforced at CI validation)

4.2 CI/CD Validation Hooks

GitHub Actions workflow validates all workflow code against grammar schema.

Validation Steps:

- Scan all Rust/Lua/ALN files for `AlnNormId`, `BioticTreatyId`, `MicroTreatyId` references
- Verify each reference resolves to active (non-deprecated) atom in registry
- Ensure no ad-hoc rights/treaty structures (e.g., inline strings, hard-coded thresholds)
- Confirm Birth-Sign binding present for any action touching land/water/air/biosignals
- Check that optimization engines import constraint compilation utilities
- Validate FPIC hooks present for workflows intersecting Indigenous territories

Sample CI Job (`.github/workflows/aletheion-governance-schema-preflight.yml`):

name: Governance Schema Preflight

on:
pull_request:
branches: [main]
push:
branches: [main]

jobs:
validate-aln-compliance:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3

```
- name: Install ALN Schema Validator
  run: |
    cargo install aln-schema-validator --locked

- name: Validate Rights Grammar Imports
  run: |
    aln-schema-validator validate-imports \
      --grammar-registry aletheion/governance/aln/ALE-GOV-RIGHTS-TREATY
      --source-dirs aletheion/erm,aletheion/governance,aletheion/trust,alethe
      --strict

- name: Check Birth-Sign Binding
  run: |
    aln-schema-validator check-birthsign-binding \
      --workflow-dirs aletheion/erm \
      --required-for land,water,air,biosignals,citizens

- name: Verify No Ad-Hoc Treaty Structures
  run: |
    aln-schema-validator scan-adhoc-structures \
      --disallowed-patterns "treaty_id.*:.*String","rights.*:.*Vec<String>" \
      --source-dirs aletheion/

- name: FPIC Hook Validation
  run: |
```

```
aln-schema-validator check-fpic-hooks \
  --workflows aletheion/erm/water,aletheion/erm/mobility,aletheion/erm/
```

Failure Modes:

- PR blocked if validation fails
- GitHub comment auto-posted with specific violation details and remediation guidance
- Workflow-specific errors tagged to responsible team via CODEOWNERS

Part V: Human-Readable Translation Layer

5.1 Citizen Interface Requirements

Stage 7 (Citizen Surface) MUST present governance context in plain language, not raw ALN syntax.

Translation Requirements:

- Every RightsAtom has companion `human_readable_statement` field
- BioticTreaties include `species_common_name` and `why_this_matters` explanatory text
- MicroTreaties store original citizen-authored policy text alongside ALN translation
- FPIC requirements link to public consultation documents and contact information

Example Translation (BioticTreaty → Citizen Notification):

ALN (Machine):
BioticConstraint {
metric: "nocturnal_scotopic_lux",
threshold: 0.5,
operator: "<=",
spatial_scope: TileSet(["zone_A"]),
temporal_scope: TimeWindow("21:00-05:00")
}

Plain Language (Human):

> **Sonoran Desert Tortoise Light Protection**
> To protect endangered tortoises during nesting season, outdoor lighting in South Mountain Preserve areas must stay below 0.5 lux between 9 PM and 5 AM from March through October. This helps tortoises navigate safely and reduces disruption to their natural behaviors.
> **Why This Matters:** Bright lights disorient tortoises and can lead them away from shelter, increasing predation risk and heat exposure. This treaty was co-developed with the Sonoran Desert Conservancy and approved by Phoenix City Council in 2026.
> **Your Actions:** If you're planning outdoor lighting installations in this area, please submit designs to the Environmental Review Office 30 days before installation. Low-intensity amber lighting is permitted with approval.

5.2 Grievance and Amendment Pathways

Citizens can challenge governance atom applications via structured grievance system.

Grievance Process:

1. Citizen submits grievance via Stage 7 app interface
2. System identifies relevant governance atoms (AlnNormId, BioticTreatyId, MicroTreatyId)
3. Grievance routed to appropriate body:
    - RightsAtom violations → LexEthos Rights Ombudsman
    - BioticTreaty disputes → Environmental Review Board
    - MicroTreaty conflicts → Neighborhood Council or DisputeCoolingProtocol mediator
    - FPIC concerns → Direct communication with Indigenous authority
4. Mediation process (14-30 day window depending on severity)
5. Resolution outcomes:
    - Uphold: Original decision stands, explanation provided
    - Amend: Governance atom updated, decision re-evaluated

    - Override: Emergency override granted with audit trail and time limit
    - Escalate: Elevated to City Council or Tribal Council for policy review
6. All grievances and outcomes appended to Googolswarm for transparency

Amendment Proposal Process:

1. Citizen or collective proposes amendment via governance portal
2. Proposal includes: target atom ID, proposed changes, justification, supporting evidence
3. Public comment period (minimum 14 days)
4. Technical review by relevant domain experts (e.g., ecologists for BioticTreaty, legal scholars for RightsAtom)
5. Vote via liquid democracy or direct ballot (depending on atom type and scope)
6. If approved: Multi-sig attestation, registry update, 30-day transition period before enforcement
7. All workflows automatically fetch updated grammar on next deployment cycle

---

Part VI: Interoperability and Extension Patterns

6.1 Cross-Workflow Inheritance

Workflows import grammar as dependency, not inline definitions.

Rust Import Pattern:

```
// In any workflow module (e.g., aletheion/erm/water/allocation.rs)
use aletheion_governance_aln::{
RightsAtom,
BioticTreaty,
MicroTreaty,
FpicRequirement,
ConsentAtom,
AlnNormId,
BioticTreatyId,
MicroTreatyId,
```

```
FpicRequirementId,
};

// Use imported types directly - never redefine locally
fn evaluate_water_allocation(
allocation_plan: &AllocationPlan,
birth_signs: &[BirthSignId],
registry: &GovernanceRegistry,
) -> Result<GovernanceEvaluation, AllocationError> {
let rights =
registry.rights_atoms.get(&AlnNormId("water_senior_rights_akimel")
);
// ... evaluation logic
}
```

ALN Import Pattern:

```
// In workflow-specific ALN file (e.g., ALE-RM-WATER-ALLOCATION-
001.aln)
import aletheion.governance.aln.RightsAtom;
import aletheion.governance.aln.BioticTreaty;
import aletheion.governance.aln.FpicRequirement;

constraint WaterAllocationConstraint {
uses: [
RightsAtom("water_senior_rights_akimel"),
BioticTreaty("riparian_flow_pulse_2026"),
FpicRequirement("SRPMIC_Water_Infrastructure_FPIC")
];
evaluation: "All imported atoms MUST evaluate to Satisfied or
HighPenalty before allocation approval";
}
```

Lua Import Pattern:

```
-- In workflow orchestration (e.g., ALE-RM-WATER-RESILIENCE-
001.lua)
local governance = require("aletheion.governance.aln")
local RightsAtom = governance.RightsAtom
local FpicStatus = governance.FpicStatus

function check_allocation_governance(allocation_plan,
birth_sign_ids)
```

local evaluation = governance.evaluate_allocation(allocation_plan, birth_sign_ids)

```
if evaluation.fpic_status == FpicStatus.Pending then
    return "PendingFpic"
elseif evaluation:has_hard_violation() then
    return "Rejected"
else
    return "Approved"
end
```

end

6.2 Domain-Specific Extensions

New workflows can extend grammar with domain-specific atom types while preserving core structure.

Extension Rules:

1. **Inherit base type**: New atom MUST extend `RightsAtom`, `BioticTreaty`, or `MicroTreaty`
2. **Preserve fields**: All core fields (holder, object, modality, conditions, expiry, enforcement_mode, provenance) MUST be present
3. **Add domain fields**: Additional fields allowed but must be scoped to prevent namespace collision
4. **Registry registration**: Extended types MUST register with governance registry before use
5. **CI validation**: Custom validators added to schema preflight CI job

Example Extension (Care Domain - Disability Accommodation Rights):

type CareRightsAtom extends RightsAtom {
// Inherited fields:
// holder, object, modality, conditions, expiry, enforcement_mode, provenance

// Care-specific extensions:
accommodation_type: "Mobility" | "Sensory" | "Cognitive" | "Chronic";
ada_compliance_level: "WCAG_AAA" | "ADA_Title_II" | "Section_508";
assistive_tech_compatibility: DeviceClass[];
care_load_impact_threshold: f64; // Max permissible CareLoadIndex increase
}

Usage in Care Workflow:
// aletheion/care/accessibility/routing.rs
use aletheion_governance_aln::RightsAtom;
use aletheion_care_aln::CareRightsAtom;

fn evaluate_mobility_routing(
route: &ProposedRoute,
user_did: &Did,
birth_signs: &[BirthSignId],
) -> Result<RoutingDecision, CareError> {
let care_atoms: Vec<CareRightsAtom> = fetch_care_rights(user_did, birth_signs)?;

```
for atom in care_atoms {
    if atom.accommodation_type == "Mobility" {
        let ada_compliant = route.check_ada_compliance(atom.ada_compliance_
        if !ada_compliant {
            return Ok(RoutingDecision::Rejected {
                reason: format!("Route fails {} compliance", atom.ada_compliance_l
                alternative_routes: compute_ada_compliant_alternatives(route, ato
            });
        }
    }
}

Ok(RoutingDecision::Approved)
```

}

Part VII: Security and Cryptographic Binding

7.1 Immutability Guarantees

All governance atoms anchored to blockchain ledger for tamper-evidence.

Googolswarm Binding:

- Each atom assigned unique content-addressed identifier: $\text{AtomId} = \text{KECCAK}(\text{serialized_atom})$
- AtomId stored in Merkle tree, root hash committed to Googolswarm block
- Any modification creates new AtomId (versioned history, not mutation)
- Multi-sig attestation required for atom registration:
  - n-of-m signature scheme: e.g., 3-of-5 City Council members + Tribal Council representative
  - Public keys registered in DID document on-chain
  - Signature verification at query time ensures authenticity
- Audit trail: All atom accesses logged with timestamp, querying workflow, decision outcome

Merkle Proof for Atom Integrity:

Given atom A with AtomId_A:

1. Workflow queries registry for atom
2. Registry returns atom + Merkle proof π_A (sibling hashes from leaf to root)
3. Workflow verifies: $\text{MerkleRoot} == \text{VerifyProof}(\text{AtomId}_A, \pi_A)$
4. If verification fails → reject decision, log security incident, trigger audit

This ensures no centralized authority can silently modify governance rules post-commitment.

7.2 DID-Based Authority Management

Governance atom updates controlled via Decentralized Identifiers (DIDs).

Authority Roles:

Role	DID Scope	Permissions
City Council Member	`did:bostrom:city_council_*`	Approve RightsAtom, MicroTreaty (city-wide)
Tribal Council	`did:bostrom:srpmic_council_*`	Approve FpicRequiremer Indigenous-scoped atoms
Neighborhood Council	`did:bostrom:neighborhood_*`	Propose/approv MicroTreaty (neighborhood scope)
Ecologist (Certified)	`did:bostrom:ecologist_certified_*`	Propose BioticTreaty, validate constraints
Citizen	`did:bostrom:citizen_*`	Propose amendments, fil grievances, vote
System Orchestrator	`did:bostrom:system_orchestrator`	Query atoms, execute prefligh checks (read-only)

Table 2: DID-based authority roles for governance atom management

Permission Verification:

```
fn verify_atom_update_authority(
atom_update: &AtomUpdate,
signer_did: &Did,
```

did_registry: &DidRegistry,
) -> Result<bool, AuthError> {
let signer_role = did_registry.resolve_role(signer_did)?;

```
match (&atom_update.atom_type, &signer_role) {
    (AtomType::RightsAtom, Role::CityCouncilMember) => Ok(true),
    (AtomType::FpicRequirement, Role::TribalCouncil) => Ok(true),
    (AtomType::BioticTreaty, Role::EcologistCertified) => Ok(true),
    (AtomType::MicroTreaty, Role::NeighborhoodCouncil) if atom_update.scop
    _ => Err(AuthError::InsufficientPermissions {
        required: atom_update.required_role(),
        actual: signer_role,
    }),
}
```

}

Multi-Sig Enforcement:

For high-stakes atoms (e.g., city-wide BioticTreaty affecting multiple species):

- Require signatures from: City Ecologist + Environmental Board Chair + 3 City Council Members + Representative from affected Indigenous nation (if applicable)
- Threshold: $\geq$ 5-of-7 signatures
- Each signature includes timestamp and justification hash
- All signatures stored in atom's `provenance.multisig_attestors` field

Part VIII: Performance and Caching Strategies

8.1 Query Optimization

Governance preflight checks (Stage 5) run on every workflow action—performance is critical.

Caching Layers:

1. **L1 - Node-Local Cache:**
   - Each edge node caches Birth-Sign bundles for its geographic coverage area
   - TTL: 1 hour (balance freshness vs. query load)
   - Invalidation: Push-based on registry updates
2. **L2 - Regional Aggregator:**
   - Cluster of nodes share cached atom lookups
   - Reduces redundant blockchain queries for same BirthSignId
   - TTL: 15 minutes
3. **L3 - Blockchain State:**
   - Authoritative source, queried on cache miss
   - Indexed by: AtomId, BirthSignId, TerritoryId, Did
   - Average query latency: <50ms

Pre-Compilation:

For frequently-used atoms (e.g., city-wide water rights, major BioticTreaties):

- Pre-compile ALN predicates to native code (LLVM IR or WebAssembly)
- Store compiled binaries alongside atom metadata
- Execute via JIT at runtime (10-100× faster than interpreter)

Benchmark Targets:

- Governance preflight check (all atoms for one action): <10ms p99 latency
- Birth-Sign lookup (cache hit): <1ms
- FPIC status verification: <5ms
- Full audit trail generation (for Googolswarm append): <50ms

---

8.2 Scalability Considerations

As Aletheion expands beyond Phoenix, governance registry scales horizontally.

Sharding Strategy:

- Partition atoms by geographic region (BirthSignId prefix)
- Each region's atoms stored in dedicated shard
- Cross-region queries use federated query protocol
- Example: `birth_sign_id.starts_with("phoenix_az")` → Phoenix shard

Eventual Consistency:

- Atom updates propagate via gossip protocol (5-second typical propagation)
- Critical updates (e.g., emergency FpicRequirement revocation) use priority broadcast
- Conflict resolution via timestamp + multi-sig authority hierarchy

Load Balancing:

- Read-heavy workload: 99% queries, 1% updates
- Deploy read replicas with 10ms sync lag tolerance
- Write operations (atom registration) batched hourly except emergencies

---

Part IX: Migration and Backward Compatibility

9.1 Versioning Scheme

Grammar registry follows semantic versioning: `MAJOR.MINOR.PATCH`

Version Bump Rules:

- **MAJOR**: Breaking changes (e.g., removing core field, changing enforcement semantics)

- **MINOR**: Backward-compatible additions (e.g., new atom type, new optional field)
- **PATCH**: Bug fixes, clarifications (no schema changes)

Current Version: `1.0.0` (this document defines v1.0.0)

Compatibility Guarantee:

- Workflows built against v1.x.y work with v1.x+1.z (forward-compatible within major version)
- MAJOR version changes require workflow updates within 90 days (deprecation grace period)

9.2 Legacy System Integration

For integrating with non-Aletheion governance systems (e.g., existing city databases, tribal registries):

Adapter Pattern:

// aletheion/governance/adapters/legacy_fpic_adapter.rs
pub struct LegacyFpicAdapter {
external_api_endpoint: String,
auth_token: String,
}

impl LegacyFpicAdapter {
pub async fn fetch_fpic_status(
&self,
territory: &IndigenousTerritoryId,
action_description: &str,
) -> Result<FpicStatus, AdapterError> {
let response = self.call_external_api(territory,
action_description).await?;

```
// Map external format to ALN FpicStatus
let aln_status = match response.status_code {
    "APPROVED" => FpicStatus::Granted {
        grant: FpicGrant {
            grant_id: FpicGrantId(response.approval_id),
```

```
                granted_at: parse_timestamp(&response.approval_date)?,
                // ... map remaining fields
            },
        },
        "PENDING" => FpicStatus::Pending { /* ... */ },
        "DENIED" => FpicStatus::Denied { /* ... */ },
        _ => return Err(AdapterError::UnknownStatus(response.status_code)),
    };

    Ok(aln_status)
}
```

}

Synchronization Protocol:

- Nightly batch sync: Import external registry updates into ALN governance registry
- Real-time sync: For time-sensitive updates (e.g., emergency FPIC revocation)
- Conflict resolution: External source is authoritative for Indigenous/tribal governance; city systems authoritative for municipal ordinances

---

Part X: Compliance with ALN/KYC/DID and Quantum Cryptographic Governance

10.1 Quantum-Resistant Signatures

Future-proof governance atoms against quantum computing threats.

Signature Schemes:

- Current: Ed25519 (classical elliptic curve)
- Migration target (2027): CRYSTALS-Dilithium (NIST PQC standard)
- Hybrid approach during transition: Dual-sign atoms with both Ed25519 and Dilithium

Transition Plan:

1. **2026 Q2**: Add Dilithium signature field to atom schema (optional)
2. **2026 Q4**: All new atoms MUST include both signatures
3. **2027 Q2**: Validation logic requires Dilithium signature verification
4. **2027 Q4**: Ed25519 signatures deprecated (kept for historical verification only)

Backward Compatibility:

- Old atoms retain Ed25519 signatures indefinitely (audit trail preservation)
- New validation logic verifies whichever signature(s) present in atom's provenance

---

10.2 DID Binding and KYC Compliance

All governance atom interactions tied to verified DIDs.

DID Requirements:

- Holders, proposers, attestors MUST have DID registered in Aletheion DID registry
- DID documents include: Public key, authentication methods, service endpoints, KYC attestation level
- KYC levels:
  - **Level 0**: Anonymous (public participation, proposals only)
  - **Level 1**: Verified resident (voting rights, grievance filing)
  - **Level 2**: Verified authority (neighborhood council, certified professionals)
  - **Level 3**: Government official (city/tribal council, law enforcement)

KYC Verification:

- Performed by licensed identity providers (government agencies, tribal enrollment offices)
- Zero-knowledge proofs for privacy: Prove "I am Level 1+ resident" without revealing full identity

- Attestations stored on-chain, revocable by issuer

Example DID Document (Neighborhood Council Member):


```
{
"id": "did:bostrom:18sd2ujv24ual9c9pshtxys6j8knh6xaead9ye7",
"authentication": [
{
"type": "Ed25519VerificationKey2020",
"publicKeyMultibase":
"z6MkpTHR8VNsBxYAAWHut2Geadd9jSwuBV8xRoAnwWsdvktH"
}
],
"service": [
{
"type": "GovernanceAuthority",
"serviceEndpoint": "https://roosevelt-council.phoenix.gov/api"
}
],
"kyc": {
"level": 2,
"attestor": "did:bostrom:city_kyc_authority",
"jurisdiction": "Roosevelt_Historic_District",
"expires": "2027-03-08T00:00:00Z"
}
}
```

Part XI: Summary and Integration Checklist

This grammar provides the complete foundation for machine-enforceable governance across all Aletheion workflows. **No workflow should define rights, treaties, or obligations outside this schema.**

Integration Checklist for Workflow Developers:

1. Import governance grammar: `use aletheion_governance_aln::*`
2. Identify spatial scope: Determine which Birth-Signs apply to workflow actions
3. Query governance registry: Fetch applicable atoms for each action

4. Evaluate constraints: Call centralized compliance runtime (Stage 5)
5. Handle outcomes:
    - Approved → Proceed to actuation (Stage 6)
    - Rejected → Log reason, notify citizen, suggest alternatives
    - PendingFpic → Queue action, initiate FPIC consultation, notify tribal authority
    - ApprovedWithDerating → Actuate with modified parameters (e.g., reduced capacity)
6. Append provenance: Record AlnNormId list + evaluation outcome to Googolswarm (Stage 3)
7. Surface to citizens: Display applicable treaties + plain-language explanations (Stage 7)
8. CI validation: Ensure GitHub Actions preflight job passes before merge

Key Governance Principles:

- **Rights as First-Class Objects**: Not comments or documentation—enforceable code
- **Territorial Binding**: Every action knows its jurisdictional context via Birth-Signs
- **Multi-Stakeholder Respect**: Indigenous, ecological, neighborhood voices encoded equally
- **Transparency**: All decisions auditable, all governance rules public
- **Contestability**: Citizens can challenge, propose amendments, file grievances
- **Immutability with Evolution**: Blockchain-anchored history + versioned updates
- **No Bypass**: No emergency override without audit trail and time-bound justification

---

Appendix A: Glossary of Terms

ALN (Aletheion Legal Norm): Machine-readable grammar for encoding legal, ethical, and ecological obligations as executable constraints.

Birth-Sign: Regional governance signature bundling laws, treaties, Indigenous protocols, and ecological protections for a spatial tile.

BioticTreaty: Cross-species governance agreement protecting non-human lifeforms (habitat, light, noise, chemical, thermal constraints).

ConsentAtom: Biosignal governance primitive with explicit revocation paths and CryptoSomatic threshold enforcement.

CryptoSomatic Shield: Real-time biosignal monitoring system enforcing FEAR-PAIN-SANITY thresholds for augmentation safety.

Did (Decentralized Identifier): W3C-standard cryptographic identity (e.g., `did:bostrom:18sd2uj...`), used for KYC, consent, and authority verification.

FPIC (Free, Prior, and Informed Consent): Indigenous governance protocol requiring explicit tribal approval before actions on traditional lands.

GovernedDecisionEnvelope: Transaction record linking workflow actions to Birth-Signs, ALN norms, DIDs, and outcomes for Googolswarm provenance.

Googolswarm: Blockchain ledger for immutable audit trails (multi-sig attestation, content-addressed governance atoms).

LexEthos: Rights grammar compiler for neighborhood-scale micro-treaties and dispute resolution protocols.

MicroTreaty: Hyper-local governance agreement (neighborhood cooling norms, workplace augmentation rules, school safety zones).

ProvenanceRef: Cryptographic link to source document (treaty, ordinance, study) verifying governance atom authenticity.

RightsAtom: Minimal unit of obligation/permission/prohibition with holder, object, modality, conditions, and enforcement mode.

SomaticEnvelope: FEAR-PAIN-SANITY threshold profile for biosignal safety (augmentation preemptive shutdown triggers).

Synthexis: Ecological synthesis engine integrating BioticTreaties into optimization constraints.

Appendix B: Contact and Governance Authorities

City of Phoenix Governance

- Office of Environmental Programs: `did:bostrom:phoenix_env_office`
- City Council Clerk: `council-clerk@phoenix.gov`
- Heat Equity Office: `did:bostrom:phoenix_heat_equity`

Indigenous Nations (Phoenix Region)

- Salt River Pima-Maricopa Indian Community: `water.resources@srpmic-nsn.gov` | `did:bostrom:srpmic_council`
- Gila River Indian Community: `naturalresources@gric.nsn.us` | `did:bostrom:gric_council`
- Fort McDowell Yavapai Nation: `tribalcouncil@ftmcdowell.org` | `did:bostrom:fmyn_council`

Ecological Stewardship

- Sonoran Desert Conservancy: `governance@sonorandesert.org` | `did:bostrom:sdc_certified_ecologist`
- Audubon Arizona: `policy@audubon.org` | `did:bostrom:audubon_az`

Citizen Governance

- LexEthos Rights Ombudsman: `ombudsman@aletheion.phoenix.gov` | `did:bostrom:lexethos_ombudsman`
- Neighborhood Council Directory: `neighborhoods@phoenix.gov`

Technical Support

- ALN Schema Maintainers: `aln-maintainers@aletheion.dev`
- Compliance Runtime Issues: `compliance@aletheion.dev`
- GitHub Repository: `https://github.com/Doctor0Evil/Aletheion`

Document Version: 1.0.0
Publication Date: 2026-03-08
Next Review: 2027-03-08
Maintainer: Aletheion Governance Working Group
License: Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)
Blockchain Anchor: bostrom:tx:ALE-GOV-RIGHTS-TREATY-GRAMMAR-001:v1.0.0:hash:<KECCAK256_PLACEHOLDER>

END OF DOCUMENT